

What is OS?

- Interface between end user and computer hardware.
- Interface between programs and computer hardware.
- Control program that controls execution of all other programs.
- Resource manager/allocator that manages all hardware resources.

Bootable CD/DVD = Core OS + Applications + Utilities
Core OS = Kernel

- Kernel performs all basic functions of the OS.
-

OS Functions

- CPU Scheduling
 - Process Management
 - Memory Management
 - File & IO Management
 - Hardware Abstraction
 - User Interfacing
 - Security & Protection
 - Networking
-

Process

- Program under execution.
- Process executes in RAM.
- Process Control Block (PCB) contains information about the process required for its execution.

PCB Contents

Process ID

- Unique identifier of the process.

Exit Status

- 0 → Indicates successful execution.
- Non-zero → Indicates failure.

Scheduling Information

- State
- Priority
- Scheduling algorithm
- CPU time information

Memory Information

- Base & Limit registers
- Segment table
- Page table

File Information

- Open files
- Current directory

IPC Information

- Signals
- Other IPC resources

Execution Context

- Values of CPU registers

Kernel Stack

- Each process has a kernel stack.

Other Names of PCB

- PCB is also called:
 - Process Descriptor (PD)
 - uarea (UNIX)
 - task_struct (Linux)
- In Linux, size of `task_struct` is approximately 4 KB.

Multi-tasking OR Time-sharing

- CPU time is shared among multiple processes in the main memory. This is called **multi-tasking**.
- In such systems, a small amount of CPU time is given to each process repeatedly, so that response time for any process is typically less than 1 second.
- With this mechanism, multiple tasks that are ready for execution can execute concurrently.

Types of Multi-tasking

Process-based Multi-tasking

- Multiple independent processes execute concurrently.
- Processes running on multiple processors are called **multi-processing**.

Thread-based Multi-tasking OR Multi-threading

- Multiple parts/functions in a process execute concurrently.

Thread Concept

- Threads are used to execute multiple tasks concurrently in the same program/process.
- A thread is a light-weight process.
- For each thread, a new control block and stack are created.
- Other sections (text, data, heap, etc.) are shared with the parent process.
- Inter-thread communication is much faster than inter-process communication.
- Context switch between two threads in the same process is faster.
- Thread stack is used to create function activation records of the functions called/executed by the thread.

Process vs Thread

- In modern operating systems, a process is a container holding resources required for execution, while a thread is the unit of execution/scheduling.
- Process holds resources like:
 - Memory
 - Open files
 - IPC resources (signals, shared memory, pipes, etc.)

PCB Contains

- PID
- Exit status
- Open files
- Signals / IPC information
- Memory information

CPU Time Allocation

- CPU time is allocated to threads.
- Thread is the unit of execution.

TCB Contains

- TID (Thread ID)

- Scheduling information
 - Priority
 - Scheduling algorithm
 - Time left
- Execution context
- Kernel stack

Main Thread

- For each process, one thread is created by default called the **main thread**.
- The main thread executes the entry-point function of the process.
- The main thread uses the process stack.

Main Thread Termination

- When the main thread is terminated, the process is terminated.
 - When a process is terminated, all threads in the process are terminated.
-

Quick Comparison: Process vs Thread

Feature	Process	Thread
Definition	Program under execution	Unit of execution
Resource Ownership	Yes	No
Address Space	Separate	Shared
Communication	IPC required	Shared memory
Context Switch	Slower	Faster
Memory Usage	More	Less
Control Block	PCB	TCB
Scheduling Unit	No	Yes

Important Definitions

Operating System (OS)

Interface between user/application programs and computer hardware.

Process

A program under execution.

Thread

A light-weight process and the smallest unit of execution.

PCB

Data structure containing all information required for process execution and management.

TCB

Data structure containing all information required for thread execution and scheduling.

Multi-tasking

Sharing CPU time among multiple processes.

Multi-threading

Executing multiple threads concurrently within the same process.

Multi-processing

Executing processes simultaneously on multiple processors/cores.

Scheduler

- Scheduler is a component of the Operating System responsible for deciding which task/process/thread should run on the CPU.
 - Since there is only one CPU (or a limited number of CPUs), multiple tasks cannot execute simultaneously on the same CPU.
 - The scheduler allocates CPU time among multiple ready tasks.
 - Scheduler helps achieve multitasking and efficient CPU utilization.
-

Why Scheduler is Required?

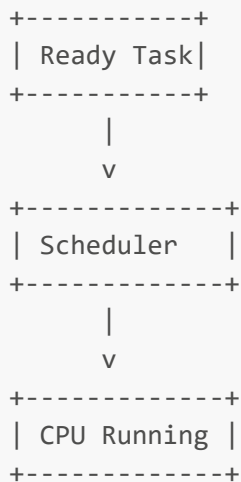
- Multiple tasks may be ready to execute at the same time.
 - CPU can execute only one task at a time on a single core.
 - Scheduler decides:
 - Which task should run?
 - How long should it run?
 - When should it be switched out?
-

Scheduling Goals

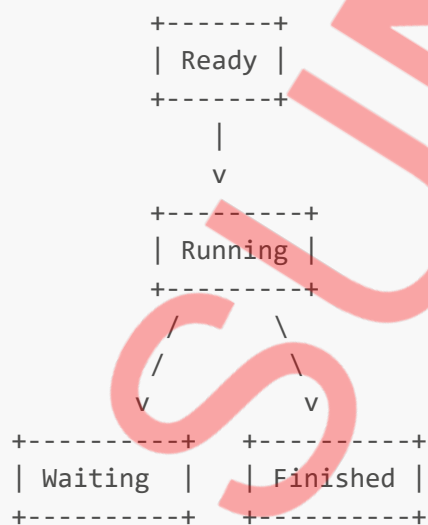
- Maximum CPU utilization

- Minimum response time
- Minimum waiting time
- Maximum throughput
- Fair allocation of CPU time
- Meet real-time deadlines (in RTOS)

Basic Scheduling Process



Process States



Types of Scheduler

Long-Term Scheduler

- Selects jobs from secondary storage.
 - Loads processes into memory.
 - Controls degree of multiprogramming.
-

Short-Term Scheduler

- Selects a process from the ready queue.
 - Allocates CPU to the selected process.
 - Runs very frequently.
-

Medium-Term Scheduler

- Temporarily removes processes from memory.
 - Swaps processes out and back in.
-

Scheduling Algorithms

FCFS (First Come First Serve)

- Process arriving first gets CPU first.
 - Simple implementation.
 - May suffer from convoy effect.
-

SJF (Shortest Job First)

- Process with shortest execution time runs first.
 - Provides minimum average waiting time.
 - Difficult to predict burst time.
-

Priority Scheduling

- Higher priority task gets CPU first.
 - Can be:
 - Preemptive
 - Non-preemptive
-

Round Robin

- Each process gets a fixed time slice (quantum).
- Common in time-sharing systems.
- Fair CPU allocation.

Example:

```
Task A -> 1ms
Task B -> 1ms
Task C -> 1ms
Task A -> 1ms
...
```

Context Switching

- Switching CPU from one task to another is called context switching.

Steps

Save Current Task Context

- Program Counter (PC)
- Stack Pointer (SP)
- CPU Registers
- Status Registers

Restore Next Task Context

- Program Counter
- Stack Pointer
- CPU Registers
- Status Registers

```
Task A
|
Save Context
|
Scheduler
|
Restore Context
|
Task B
```

Scheduler in RTOS

- RTOS scheduler generally uses priority-based scheduling.
- Highest priority ready task gets CPU.
- If a higher priority task becomes ready, current task may be preempted.

Example:

```
Task A Priority = 3
Task B Priority = 2
Task C Priority = 1
```

Execution:

```
Task A runs first
If Task A blocks,
Task B runs
If Task B blocks,
Task C runs
```

Timer and Scheduler

- Scheduler requires a periodic event to decide when task switching should occur.
- This periodic event is generated by a hardware timer.

Why Timer is Required?

Without timer:

```
Task A runs forever
```

Scheduler never gets control.

With timer:

```
Task A
|
Timer Interrupt
|
Scheduler
|
Task B
```

Hardware Timer

- Timer is a hardware peripheral.
- Timer counts clock pulses.
- When count reaches a programmed value, an interrupt is generated.

Example:

CPU Clock = 100 MHz

Timer Period = 1 ms

Every 1 ms:

Timer Interrupt Generated

System Tick

- Periodic timer interrupt used by scheduler is called System Tick.
- Also called Tick Interrupt.

Example:

1 Tick = 1 ms

Then:

1000 Ticks = 1 Second

Scheduler Operation Using Timer

```

Timer Interrupt
  |
  v
Tick Count Increment
  |
  v
Scheduler Executes
  |
  v
Select Next Task

```

|
v
Context Switch

Advantages of Timer-Based Scheduling

- Fair CPU sharing
- Supports multitasking
- Enables task delays
- Enables timeouts
- Supports preemptive scheduling
- Improves system responsiveness

Interview Questions

What is a Scheduler?

Scheduler is an OS component that decides which task/process/thread should get CPU time.

Why is a Timer Required for Scheduling?

Timer generates periodic interrupts that allow the scheduler to regain control of the CPU and decide whether a task switch is required.

What is a System Tick?

A periodic timer interrupt used by the scheduler for time management and task switching.

What is Context Switching?

Saving the current task's execution context and restoring another task's context so that CPU execution can switch between tasks.

Real-Time Systems

- The Operating System in which accuracy of results depends on both:
 - Accuracy of the computation
 - Time duration in which the results are produced

is called a **Real-Time Operating System (RTOS)**.

- In real-time systems, producing correct results is not sufficient.

- Results must also be produced within a specified time limit (deadline).

Real-Time Operating System (RTOS)

Definition

- RTOS is an Operating System designed to guarantee that tasks are completed within a definite time duration.
- The correctness of a system depends on:
 - Logical correctness of computation
 - Timing correctness of computation

Deadline

- The maximum allowed time for completing a task is called a **deadline**.

Example:

```
Sensor detects obstacle
      |
      v
Brake command generated within 10 ms
```

- If the brake command is generated after 10 ms, the system may fail even if the computation is correct.

Importance of Timing

In a normal Operating System:

Correct Result + Late Response = Acceptable

In a Real-Time Operating System:

Correct Result + Late Response = Failure

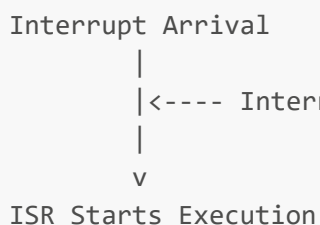
Characteristics of RTOS

- Deterministic behavior
- Predictable execution
- Fast response time
- Small interrupt latency
- Small context switch time
- Priority-based scheduling
- High reliability
- Supports multitasking

Interrupt Latency

Definition

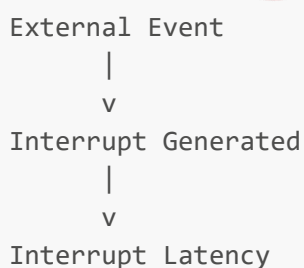
- Time from the arrival of an interrupt until the processor starts executing the corresponding Interrupt Service Routine (ISR).

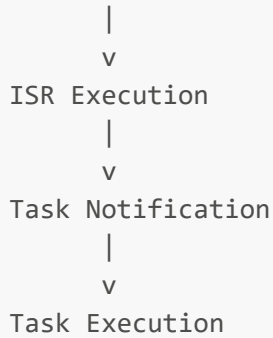


Why Interrupt Latency is Important?

- Real-time systems must respond quickly to external events.
- Large interrupt latency may cause deadlines to be missed.
- RTOS aims to keep interrupt latency:
 - Small
 - Predictable
 - Fixed (deterministic)

RTOS Interrupt Handling





Deterministic Behavior

- RTOS provides predictable timing behavior.
- Execution time of critical operations can be estimated accurately.

Example:

```

Task Execution Time = 5 ms
Interrupt Latency   = 10 us
Context Switch Time = 20 us
  
```

- These values remain nearly constant.

RTOS Scheduling

- RTOS generally uses priority-based scheduling.
- Highest priority ready task gets CPU first.

Example:

```

Task A Priority = 3
Task B Priority = 2
Task C Priority = 1
  
```

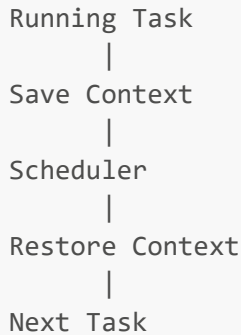
Execution Order:

```

Task A
Task B
Task C
  
```

Context Switching in RTOS

- RTOS performs fast context switching.
- Context switching time is usually fixed and predictable.



Types of Real-Time Systems

Hard Real-Time System

- Missing a deadline causes system failure.
- Deadlines must always be met.

Examples:

- Airbag Control System
- Aircraft Flight Control
- Nuclear Reactor Control
- Anti-lock Braking System (ABS)

Soft Real-Time System

- Missing a deadline degrades performance.
- Occasional deadline misses are acceptable.

Examples:

- Video Streaming
- Audio Processing
- Online Gaming

Firm Real-Time System

- Results produced after deadline are useless.
- Occasional misses are tolerated.

Examples:

- Industrial Inspection Systems
 - Radar Tracking Systems
-

Applications of RTOS

Automotive

- Engine Control Unit (ECU)
- ABS
- Airbag Systems

Industrial Automation

- PLC Systems
- Motor Control

Aerospace

- Flight Control Systems
- Navigation Systems

Medical Devices

- Patient Monitoring Systems
- Infusion Pumps

Consumer Electronics

- Smart Appliances
 - IoT Devices
-

Important Definitions

Real-Time System

A system in which correctness depends on both the result and the time at which the result is produced.

RTOS

An Operating System that guarantees task completion within specified timing constraints.

Deadline

The maximum allowable time to complete a task.

Interrupt Latency

Time between interrupt occurrence and start of ISR execution.

Deterministic Behavior

Ability of a system to provide predictable execution times.

Interview Questions

What is a Real-Time Operating System?

An RTOS is an Operating System that guarantees tasks will complete within specified deadlines.

What is Interrupt Latency?

The time from the arrival of an interrupt until the processor starts executing its ISR.

Why is Interrupt Latency Important in RTOS?

Because high interrupt latency can cause deadlines to be missed, leading to system failure.

What is the difference between RTOS and General Purpose OS?

RTOS focuses on predictable timing and meeting deadlines, while a general-purpose OS focuses on throughput, fairness, and overall performance.

GPOS vs RTOS

Similarities Between GPOS and RTOS

1. Hardware Abstraction

- Both provide hardware abstraction.
 - Applications access hardware through OS services rather than directly interacting with hardware.
-

2. Multi-tasking Support

- Both provide some level of multi-tasking.
 - Multiple tasks can execute concurrently by sharing CPU time.
-

3. Resource Management

- Both manage system resources such as:
 - CPU
 - Memory
 - I/O devices
 - Files
 - Network resources
-

4. System Services

- Both provide system services (system calls/APIs) for applications.
 - Applications use these services to access OS functionality.
-

Differences Between GPOS and RTOS

1. Customization Support

GPOS

- Little or no customization support.
- Linux is an exception because it can be customized.

RTOS

- Fully customizable.
 - Can be configured to include only required components.
 - Suitable for systems with limited memory.
-

2. Interrupt Latencies

GPOS

- Higher interrupt latencies.
- Typically in milliseconds (ms).
- Interrupt response time may vary depending on system load.

RTOS

- Lower interrupt latencies.
 - Typically in microseconds (μ s).
 - Interrupt handling is deterministic and predictable.
-

3. Deterministic Behavior

GPOS

- Non-deterministic.
- Execution time varies depending on system activity.

RTOS

- Deterministic.
 - Execution and response times are predictable.
-

4. Memory Requirements

GPOS

- Requires more memory.
- Larger memory footprint due to:
 - Processes
 - IPC mechanisms
 - Drivers
 - File systems

RTOS

- Requires less memory.
 - Designed for low-end embedded systems.
 - Minimal memory footprint.
-

5. Disk Support

GPOS

- Full support for secondary storage devices.
- Supports:
 - HDD
 - SSD
 - USB drives
 - Optical drives

RTOS

- Little or no support for secondary storage.
 - Storage support is optional.
 - Many embedded systems operate without disks.
-

6. CPU Architecture Support

GPOS

- Supports multiple architectures.

Examples:

- x86
- x86_64
- ARM
- RISC-V
- PowerPC

RTOS

- Usually supports a limited number of architectures.
 - Depends heavily on hardware-specific features.
-

7. Scheduling Policies

GPOS

Supports:

Real-Time Policies

- FIFO
- Round Robin (RR)

Non Real-Time Policies

- NORMAL
- BATCH
- IDLE

RTOS

Supports only real-time scheduling policies.

Examples:

- FIFO
 - Round Robin (RR)
 - Priority-Based Scheduling
-

8. Timer Management

GPOS

- Timer hardware may be:

- Fine-grained (1 ms)
- Coarse-grained (10 ms or more)
- Timer frequency is generally configured during boot.
- Timer configuration is usually not changed dynamically.

RTOS

- Requires fine-grained timers.
- Typical tick period:

1 ms

- Timer is re-programmable.
- Supports:

One-Shot Mode

Interrupt generated only once.

Periodic Mode

Interrupt generated repeatedly.

- Special APIs are provided to configure timers.

9. Interrupt Management

GPOS

Interrupt processing is divided into two parts:

Top Half

- Executed immediately.
- Non-blocking.
- Performs critical work.

Bottom Half

- Executed later.
- Can perform deferred processing.

Purpose:

Reduce interrupt latency.

RTOS

- ISRs are minimal and non-blocking.
- ISR performs only critical operations.
- Further processing is typically deferred to a high-priority task.
- Interrupt-related tasks execute before lower-priority tasks.

10. I/O Subsystem and Device Drivers

GPOS

- Drivers interact through multiple OS layers.
- Driver execution may involve:
 - Scheduler
 - VFS
 - IPC
 - Memory management

Result:

Higher latency.

RTOS

- Drivers and tasks usually execute in the same address space.
- Minimal software layers.
- Faster response and lower latency.

11. Task Management

GPOS

- Processes and threads are heavy-weight.
- Higher memory consumption.
- Larger management overhead.

RTOS

- Tasks are light-weight.
- Lower memory requirements.
- Faster creation and deletion.

Summary Table

Feature	GPOS	RTOS
Purpose	General Computing	Real-Time Applications
Timing Guarantee	No	Yes
Customization	Limited	Highly Customizable
Determinism	No	Yes
Interrupt Latency	Higher (ms)	Lower (μ s)
Memory Footprint	Large	Small
Disk Support	Full	Limited/Optional
CPU Architecture Support	Wide	Limited
Scheduling	Real-Time + Non Real-Time	Real-Time Only
Context Switch Time	Variable	Predictable
Goal	Maximize Throughput	Meet Deadlines
Scheduling Fairness-Based		Priority-Based
Timer Management	Fixed	Reprogrammable
Interrupt Handling	Top Half + Bottom Half	Minimal ISR + High Priority Tasks
Driver Latency	Higher	Lower
Task Size	Heavy-weight	Light-weight

Examples of GPOS

- Linux
- Windows
- macOS
- Android

Examples of RTOS

- FreeRTOS
- VxWorks
- uC/OS
- pSOS
- RTLinux

- ThreadX
- Zephyr
- QNX

Important Interview Questions

What is the major difference between GPOS and RTOS?

GPOS focuses on maximizing throughput and user experience, whereas RTOS focuses on meeting timing deadlines and providing deterministic behavior.

Why does RTOS have lower interrupt latency?

Because RTOS uses minimal ISRs, deterministic scheduling, and fewer software layers, resulting in faster interrupt response.

Why are RTOS tasks called light-weight?

Because they require less memory and have lower management overhead compared to processes and threads in a GPOS.

Why is deterministic behavior important in RTOS?

Because missing a deadline can cause system failure even if the computation result is correct.